Binance — security stack + in-house fingerprint assessment (2026-10-31)

Subject: what protects `binance.com` and what we can learn from it
Method: public-site analysis + runtime observation. All 3rd-party telemetry observed or fulfilled locally; no account, no auth, nothing forwarded.
Result: Binance runs a **layered stack**: AWS WAF (outer gate) → **in-house `BNC_FV_KEY` device fingerprint** (client-computed, tamperable) → Themis feature-flag service → HMAC/2FA API auth. The in-house fingerprint is the same "client-computes-the-proof" pattern we've cracked across this project — but the WAF + API-auth layers are the real defense.

---

1. The stack (layered)

Layer	Product	Evidence
Edge WAF	AWS WAF (JS challenge)	`x-amzn-waf-action: challenge`, `fe4385362baa.…token.awswaf.com/challenge.js` (691 KB), mints `aws-waf-token`
Device fingerprint	**Binance in-house**	`dynamic-fingerprint.4a1db346.js`, `BNC_FV_KEY` cookie, `localStorage.__BNC_FP_INFO__`
Feature flags / risk	Themis (third-party)	`api.saasexch.co/bapi/themis/…` (`v2/strategy/codeless/query`, `v3/expt/query`, `v3/fg/query`)
Monitoring	Sentry	`browser.sentry-cdn.com` + `o529943.ingest.sentry.io`
Reporter	Binance in-house	`reporter-sdk@4.5.45.js` (`_bn_reporter`, `_bn_http_init`)
API auth	HMAC + 2FA	`CB-ACCESS`-style (exchange) / TOTP on high-value actions

Cookies: `aws-waf-token`, `bnc-uuid`, `BNC_FV_KEY`, `BNC_FV_KEY_T`, `BNC_FV_KEY_EXPIRE`.

2. AWS WAF layer (the outer gate)

- `challenge.js` = classic `a0_0x`-obfuscated JS challenge: collects navigator/canvas/webgl/webdriver/screen + `performance` timing, posts `mp_verify`, mints `aws-waf-token` (`uuid:region:challenge:signature`).
- Served from the per-region token host (`…token.awswaf.com`); the WAF challenge is the "are you a real browser + pass the JS proof" gate.
- **This is the same class as the Coinbase Cloudflare layer** — a transport+JS challenge, not a client-mintable token.

3. The in-house fingerprint (`BNC_FV_KEY`) — client-computed, tamperable

The engine (`dynamic-fingerprint.4a1db346.js`, 65 KB) collects:

```
fonts (mmMwWLliI0O&1 probe), domBlockers (ad-block selectors),
fontPreferences, audio (OfflineAudio hash), screenFrame, canvas (toDataURL),
osCpu, languages, colorDepth, deviceMemory, screenResolution,
hardwareConcurrency, timezone, sessionStorage, localStorage, indexedDB,
openDatabase, cpuClass, platform, webgl (getParameter + UNMASKED_*)
```

The computed fingerprint — persisted in `localStorage.__BNC_FP_INFO__`:


```
{
  "screen_resolution": "900,1280",
  "available_screen_resolution": "900,1280",
  "system_version": "Linux ",
  "brand_model": "unknown",
  "system_lang": "en-US",
  "timezone": "GMT-05:00",
  "timezoneOffset": 300,
  "user_agent": "Mozilla/5.0 … HeadlessChrome/148…",
  "list_plugin": "",
  "canvas_code": "8c6d5a13",
  "webgl_vendor": "Google Inc. (Google)",
  "webgl_renderer": "ANGLE (Google, Vulkan 1.3.0 (SwiftShader …))",
  "audio": "124.04347527516074",
  "platform": "Linux x86_64",
  "web_timezone": "America/Chicago",
  "device_name": "Chrome V148.0.7778.96 (Linux)",
  "fingerprint": "08a1a901da0a12f69df213c8a36049e4"
}
```


The cookie: `BNC_FV_KEY` = 40-hex device id, `BNC_FV_KEY_T` = timestamped signature, `BNC_FV_KEY_EXPIRE` = expiry. The server's risk-control reads this on API calls.

Tamper proof (same pattern as every vendor in this project)

	fingerprint	canvas_code	webgl_renderer
clean	`08a1a901da0a12f69df213c8a36049e4`	`8c6d5a13`	SwiftShader (headless)
tampered (screen 1920×1080, hw 99, Win32, canvas noop)	**`b7fa6b04e80d66a72dfac5aa795d9904`**	**`50758fd4`**	SwiftShader

Changing screen/CPU/platform/canvas changes the computed fingerprint → the `BNC_FV_KEY` identity is **fully client-mintable**. A client can rewrite any signal before the hash is computed. The fingerprint is evidence, not proof — the real decision is server-side.

4. The Themis layer (feature flags / risk orchestration)

`api.saasexch.co` (a third-party feature-flag service) receives `device_id`, `user_agent`, `country`, `lang` on `/bapi/fe/themis/*` — used for A/B testing and feature rollout, and as a risk input. The `device_id` here is a UUID (not the fingerprint hash) — the fingerprint rides the `BNC_FV_KEY` cookie instead.

5. What makes this "good" vs "bad" (the learning)

Good (what Binance gets right)	Bad (the fingerprint)
Layered: WAF challenge *and* API auth *and* fingerprint — no single point	**Client-computed fingerprint** = tamperable (proven above)
WAF verifies transport+JS execution (the Coinbase/CF lesson)	`__BNC_FP_INFO__` in localStorage = readable, writable
Server risk-control reads the cookie server-side	Fingerprint is a client-asserted identity, not a proof
Themis + Sentry = telemetry	Obfuscation (webpack chunks) is minimal

The honest take: Binance's *defense* is the layering — the WAF stops automated launches, the API auth stops unauthorized actions, and the fingerprint is a *correlation* signal, not the boundary. That's a better posture than the device-intel SDKs (which make the client token the whole game) but it's the same client-computes-the-proof pattern underneath.

6. Honest boundaries

- We observed the fingerprint, proved it's tamperable, and mapped the stack.
- We did **not** authenticate, did **not** bypass the AWS WAF (we ran *inside* a browser that passed it — the same "clean launch clears the gate" dynamic), did **not** forge an API call, and did **not** touch any account.
- The `BNC_FV_KEY` / `BNC_FP_INFO` are Binance's own client-side artifacts — no server compromise implied or attempted.

7. Artifacts

File	What
`assets/reporter-sdk.js`	Binance in-house reporter SDK (494 KB)
`assets/awswaf-challenge.js`	AWS WAF JS challenge (691 KB)
`assets/dynamic-fingerprint.js`	the in-house fingerprint engine (65 KB)
`scripts/bn-probe.mjs`	page probe (requests, globals, cookies)
`scripts/bn-tamper.mjs`	fingerprint tamper differential
`artifacts/probe1.json`	request/global map
`artifacts/device-info.json`	`_bne.device.getDevice()` output
`artifacts/fp-info.json`	**the full `__BNC_FP_INFO__` fingerprint**
`artifacts/fingerprint-state.json`	localStorage + cookies
`artifacts/fp-posts.json`	Themis posts (device_id, UA)

8. Summary line

> *"Binance layers AWS WAF (transport+JS challenge) with an in-house `BNC_FV_KEY` device fingerprint — a classic FingerprintJS-style client hash (canvas/webgl/audio/fonts/hardware) persisted in localStorage and shipped as a cookie. We read the full fingerprint and proved it's tamperable (change a signal → new hash). The real defense is the layering + server-side risk, not the client fingerprint."*